

Cooperative Agent Architecture — Game-Informed Design Specification

Status: Design Draft · **Phase:** 2a Extension · **Updated:** 2026-04-01 **Companion to:** ARCHITECTURE.md §14 (Bot & Agent Framework)

Table of Contents

1. [Design Thesis](#)
2. [Game Mechanic Inventory](#)
3. [Orchestration Boundary](#)
4. [Module Structure](#)
5. [Cadence System](#)
6. [Formation System](#)
7. [Momentum System](#)
8. [Awareness System](#)
9. [Attention Economy](#)
10. [Shared Environment](#)
11. [Consensus Engine](#)
12. [Synchronized Commit](#)
13. [Interference Detection](#)
14. [Role Transformation](#)
15. [Rally & Cascade Recovery](#)
16. [Dynamic Capability Binding](#)
17. [Integration with Existing Architecture](#)
18. [Recovery & Mutual Aid System](#)

19. [Communication Protocols](#)
 20. [Handoff & Transition](#)
 21. [Shared Mental Model](#)
 22. [Tempo & Pacing](#)
 23. [Specialization vs Generalization](#)
 24. [Sacrifice & Covering](#)
-

1. Design Thesis

Game developers are the most experienced practitioners of multi-agent cooperative systems in existence. Every cooperative game ships a working multi-agent architecture that has been playtested by millions of users under adversarial conditions. This document treats game designs as engineering references, not analogies.

The current `orchestrator/` module (§14.3 in ARCHITECTURE.md) models coordination as a parent→child pipeline tree derived from Clicky's subagent pattern. This is command-and-control: the parent allocates fuel, defines child capabilities, collects results. Children cannot communicate laterally, cannot self-organize, cannot recover from sibling failure.

This document specifies a `cooperation/` module that handles everything between intent and outcome. The `orchestrator/` module is preserved but scoped to its proper role. The balance between orchestration and cooperation follows the CTDE paradigm (Centralized Training with Decentralized Execution) from multi-agent reinforcement learning research — the same pattern that every successful cooperative game independently discovered.

What Orchestration Owns (and nothing more)

Composition: Which agents form a group. Patapon's army selection screen. Total War's unit recruitment. Siege's operator pick phase.

Intent: What the formation should accomplish. Not how. Patapon's drum patterns. Total War's attack/defend orders. Siege's IGL strat call.

Constraints: Boundaries that cannot be exceeded. Total War's guard mode toggle. Siege's round timer. Patapon's limited command vocabulary. The sentinel's existing controls — rate limits, circuit breakers, destructive action gates, fuel budgets.

Intervention: Recovery when cooperation breaks down. Total War's general rally. Left 4 Dead's Director spawning a Tank. The circuit breaker tripping.

What Cooperation Owns (everything else)

Task decomposition within intent. Timing coordination. Role adaptation. Information fusion. Failure recovery within the formation. Resource allocation within constraints. These are documented in §5–§16.

Evidence for the Balance

Total War's architecture is documented as three layers: unit AI (individual troop movement, formation maintenance — fully autonomous), combat AI (220 rules considering unit type, condition, size, terrain, weather, and morale — cooperative), and campaign AI (strategic resource allocation — orchestrated). The combat AI layer — where cooperation lives — draws its rules from Sun Tzu's Art of War and executes them locally per unit. The player provides strategic intent. The 220 rules provide cooperative behavior. Individual units handle micro. (Source: Tommy Thompson, "The Road To War: The AI of Total War," AI and Games / Game Developer, 2023; GamesRadar+ Shogun retrospective with Creative Assembly, 2025.)

Patapon's designer Hiroyuki Kotani explicitly described the design as two layers: "One is the very simple game, so that a player can clear the goal with just three commands. But for those users who want to play a deeper game, they can use weapons; they can collect equipment and weapons." The orchestration layer is deliberately narrow (three to four drum commands). Depth lives in the cooperation layer. (Source: Gamasutra interview, 2008.)

Kotani also explained the intentional communication constraint: "While the player can understand what the Patapons are saying, they cannot send any messages to them in any other forms but the rhythms of the drums. I believe that this restriction adds attraction to this story/game." The restriction IS the design. (Source: Siliconera interview, 2008.)

Valve's Michael Booth treated the entire Left 4 Dead survivor team as "one player" for design purposes. The AI Director controls pacing and macro difficulty. It never tells individual players what to do. It adjusts emotional intensity based on group-level metrics. "Throughout the design process we treated the entire survivor team as one player." (Source: GDC 2009; Shacknews coverage.)

2. Game Mechanic Inventory

Each game referenced in this document contributes specific mechanical systems. This section catalogs them with source attribution before mapping them to modules.

2.1 Patapon (Pyramid / Japan Studio, 2007)

Rhythm-as-intent encoding. Four drum buttons produce 4-beat command sequences: PATA PATA PATA PON (march), PON PON PATA PON (attack), CHAKA CHAKA PATA PON (defend), DON DON CHAKA CHAKA (charge). The player sets intent through rhythm. Individual Patapons interpret it through their class, equipment, HP, and environmental context. Fever mode (sustained rhythm for 10+ measures at combo 3+ with perfect timing) transforms unit behavior qualitatively — units gain attack patterns that don't exist outside Fever. (Source: Wikipedia/Fandom gameplay documentation; PlayStation Blog creator interview, 2017.)

Design-time composition, runtime cooperation. The player selects army composition (unit types, equipment) before each mission. During the mission, the ONLY input channel is the drum. No individual unit selection. No target assignment. (Source: Siliconera Kotani interview, 2008.)

Environmental context modifies all behavior. Rain extinguishes fire attacks. Wind affects arrow range. Desert heat causes faster fatigue. Miracles during Fever mode can change weather. Same rhythm + different environment = different cooperative outcome.

2.2 Total War (Creative Assembly, 2000–present)

Three-layer AI architecture. Unit AI (individual soldiers maintaining formation, filling gaps), combat AI (220 rules derived from Art of War for group-level tactical behavior), campaign AI (strategic resource allocation). The developer of Total War's AI described the three layers: "the unit AI that controls individual troops and keeps them in formation and on point, the combat AI that groups and sets formations to units and the campaign/diplomacy AI that conducts the turn-based strategy." Mike Simpson called the Empire AI "by far the most complex code edifice I've ever seen in a game" and noted that "even quite simple 'static' evaluate-act AI's with no plans or memory can be complex enough to exhibit chaotic behaviour." (Source: Tommy Thompson, AI and Games / Game Developer, Parts 1–2, 2023.)

Morale as composite local signal. Each unit's morale is continuously modified by: casualties taken, casualties inflicted, flank/rear attacks, nearby friendly routing, general proximity, fatigue level, experience level, charge state. When morale hits zero, the unit routes autonomously. Morale shock triggers when 20% casualties occur within 4 seconds. (Source: Total War Center XML documentation for Napoleon/Empire.)

Cascade failure through local propagation. Routing is contagious. Adjacent units see allies flee, take morale penalties. Low-morale units like Ashigaru "will usually rout along with everyone else" when allies break. The ONLY counter is the general's rally aura, but units whose morale is shattered will not respond to rally commands. (Source: Shogun 2 encyclopedia; GameFAQs Shogun guide by Proudnerd.)

Guard mode vs free mode. A single toggle per unit that fundamentally changes

cooperative behavior. Guard mode: hold position, respond only to immediate threats. Free mode: pursue fleeing enemies but risk exposure. The appropriate setting depends on what the rest of the army is doing.

2.3 Crypt of the Necrodancer (Brace Yourself Games, 2015)

External clock as infrastructure. The beat belongs to the music, not to any player. Both players and all enemies synchronize to it. Neither player is the drummer. The clock is environmental.

Simultaneous commitment under shared timing. Both co-op players must independently choose actions that don't conflict. Two players cannot occupy the same tile. Every beat is a simultaneous decision point.

Timing forgiveness as design insight. Creator Ryan Clark: "I tried increasing this leeway value and was surprised to discover that it felt best at 100%! I think I was unwise to emulate rhythm games here. Most rhythm games require you to simply obey commands, and their challenge comes from doing that accurately. In NecroDancer the challenge comes from the fast tactical combat itself." The cooperation challenge should be choosing the RIGHT action, not hitting the timing window. (Source: Gamasutra Game Design Deep Dive, 2014.)

Co-op originally considered impossible. The SYNCHRONY DLC developer Marukyu found that the original developer had stated "real-time multiplayer was not possible due to NecroDancer's nature as a time-sensitive rhythm game." Marukyu rewrote the engine from scratch with rollback networking to make it work — showing that cooperative architecture requires purpose-built infrastructure, not retrofits. (Source: mod.io interview, 2023.)

2.4 Splinter Cell Co-op (Ubisoft Montreal, 2005–2013)

Physical interdependence through boost mechanics. One player boosts the other to an unreachable ledge. The booster is stuck until the boosted player opens a path. Sequential dependency where neither agent is the permanent "parent." (Source: GameFAQs co-op walkthrough documenting boost, rappel, hang-over, and stand-on-shoulder mechanics.)

Synchronized commit with dual breach. Both players stack on opposite sides of a door. One initiates a countdown. Both must execute simultaneously. Failure of either after commit exposes both.

Mark and execute as cooperative planning. Each player marks targets. When both have marks set, either triggers simultaneous execution. The cooperation is in the PLANNING phase. Execution is automated. (Source: GameSpot Splinter Cell rankings article, 2022.)

Asymmetric information fusion. Split approach missions: one player infiltrates from high (sees through windows), the other from ground (hears conversations, detects electronics).

Neither has complete information. (Source: Chaos Theory co-op level design documentation.)

Shared detection state. If one player is spotted, the alarm triggers for both. Mutual accountability without mutual control.

Core team lineage. Army of Two's producer Reid Schneider noted the core team came from Ubisoft Montreal's original Splinter Cell team. (Source: Unreal Engine blog interview, 2007.)

2.5 Army of Two (EA Montreal, 2008)

Explicit aggro economy. The aggro meter is a visible pendulum-like gauge. Whichever player has higher aggro draws all enemy attention. The other becomes semi-transparent. Weapon upgrades affect the aggro stat — suppressors reduce generation, flashy customizations increase it. "All of the upgrades affect the Aggro section of the weapons stats, the higher the Aggro stat is, the faster you generate Aggro when firing." (Source: P3anut Reviews weapon customization breakdown, 2008; Army of Two Wiki.)

Overkill mode from max aggro. Full aggro meter triggers fury with multiplied damage for 15 seconds. Other player becomes transparent and moves faster. Time-limited power spike from maximal cooperative role differentiation. (Source: Army of Two Wiki aggro system documentation.)

Feigned death as cooperative deception. One player fakes going down. All aggro transfers. The "downed" player crawls into flanking position undetected. (Source: Army of Two Wiki co-op mechanics.)

AI designed ground-up for cooperation. Schneider: "We added our own layer of AI and behaviors. You won't see UT or Gears types of behaviors in our AI. We've created that all from the ground up. Our AI is pretty deadly. None of it's scripted." (Source: Unreal Engine blog, 2007.)

Co-op as founding constraint. The game was designed "from the ground up" requiring both teammates to communicate and work cohesively. "Dependence on a partner (whether human or PAI) is so pronounced that most objectives are impossible to complete without it." (Source: Wikipedia; Unreal Engine blog.)

2.6 It Takes Two (Hazelight Studios, 2021)

Dynamic capability reassignment per chapter. Every chapter gives both players completely new tools. Hammer + nails. Sap gun + match gun. Size asymmetry. Time manipulation. The cooperative STRUCTURE persists (interdependent tools that chain together) while the specific TOOLS change entirely.

Hazelight's design philosophy. Josef Fares: "The number one key thing at Hazelight is

that there is a passion to what we do." On their evolution: "A more mature team, better technical tools, better understanding of design of the mechanics, we're better at writing a story for co-op, and obviously we've become better knowing what and what not to cut early." (Source: Xbox Wire interview, March 2025.)

Internal innovation process. Hazelight runs "freaky weeks" where "everybody across the team can just go crazy and do whatever they want. And it's not just the designers. It's coders, animators, whoever. It could be the cleaners in here. Everybody's welcome." (Source: EA Q&A with Fares, March 2025.)

Cooperation as the only mode. Like *A Way Out* before it, *It Takes Two* has no single-player option. It is playable only in cooperative multiplayer. The game enforced the design constraint that everything must work cooperatively.

2.7 *As Dusk Falls* (Interior Night, 2022)

Weighted consensus for narrative decisions. Up to 8 players vote on story choices simultaneously. Timer counts down. Majority wins.

Override as scarce conviction resource. "Each player can also override the other players and single-handedly choose one of the options. Simultaneously, overridden players can then override the original override." The host can configure how many overrides are available. (Source: *Lords of Gaming* interview with Marchal and Desodt, 2022; Xbox Wire FAQ, 2022.)

Design intent from inception. Marchal: "From the very beginning of the concept of the game, it's always been about taking the game to the players. Multiplayer was a pillar of doing these interactive stories." The design remained "surprisingly stable through its development: focusing on lean interactions, making it multiplayer, a Twitch mode, no game over." (Source: Microsoft Source feature, 2022.)

Consequence accumulation. Decisions in early chapters affect outcomes in later chapters. Creates shared narrative ownership where every player contributed but nobody fully controlled the outcome.

2.8 *Rainbow Six Siege* (Ubisoft Montreal, 2015–present)

Destructible environments as shared mutable workspace. "Procedural destruction plays a central role and is used to change the environment and influence the game's outcome." The core design decision: "As soon as we saw that, we were iterating on where to take the franchise. We said 'This needs to be in *Rainbow Six*.'" Level design required understanding that "the level is going to be opened up and has to make sense." (Source: MCV Develop post-mortem with Alexandre Remy, 2017.)

Emergent operator synergy. Bandit “tricks” — discovered during beta, never anticipated in production. “We never had anyone doing that during production, we had no idea Bandit could do that before release. But we love it, it has become a full part of this operator and of the game in general.” (Source: MCV Develop, 2017.)

Data-driven cooperative design. The team uses “key metrics carefully engineered to fit the issues at hand” plus pro player workshops, internal playtests across skill levels, and community feedback. (Source: GDC 2018 “Intelligent Game Design on Rainbow Six Siege,” Athanassoff and Mouret.)

Dead player as intelligence agent. When killed, players can switch to cameras and provide callouts. Primary capability lost → role transforms to information agent.

Round timer as temporal budget. 3 minutes per round. Every second spent gathering information costs execution time. Forces prioritization.

2.9 Monster Hunter (Capcom, 2004–present)

No designated roles, emergent spatial cooperation. 14 weapon types, none are “healer” or “tank.” Hammer users gravitate to the head (stun damage). Cutting weapons target the tail (sever). Cooperation is spatial and temporal — when one player topples the monster, others must already be positioned to exploit the window.

Flinch/stagger as cooperation tax. All weapons stagger teammates on contact. Wide-swing weapons like Longsword are especially disruptive. This teaches spatial positioning — experienced teams distribute around the monster rather than clustering on the same weak point. The flinch system is Monster Hunter’s version of Helldivers’ friendly fire — it punishes uncoordinated proximity and rewards spatial awareness. Newer games added a Flinch Free armor skill that some argue undermines the cooperative learning curve.

Shared failure with cart system. Three player deaths = hunt fails for everyone. Each death is a shared cost, creating tension between aggression and caution that each player resolves locally based on their HP, remaining carts, and the monster’s state.

No friction from monetization. Producer Tsujimoto explicitly stated MH would not include gameplay-affecting microtransactions because “as Monster Hunter is meant as a cooperative game, they did not want to create any type of ‘friction’ between players.” (Source: Wikipedia/Capcom interviews.) Cooperation requires a level playing field — the architectural equivalent is that agent capabilities should not be gated behind paid tiers.

Environment as cooperative medium. Director Yuya Tokuda at GDC 2018: Monster Hunter World’s “most ambitious change” was “to make it possible to use the environment.” Designed around three core pillars: compelling action, convivial multiplayer, and an engaging upgrade loop. The environments are “living ecosystems that could be taken

advantage of during hunts." (Source: Game Developer GDC 2018 MHW postmortem coverage.)

SOS flare as dynamic composition. The SOS flare system lets a hunter in difficulty broadcast a request. Other players can choose to join. This is voluntary, capability-aware formation joining — a hunter sees the quest, the monster, and the difficulty, and self-selects based on whether they can contribute.

2.10 Deep Rock Galactic (Ghost Ship Games, 2020)

Interdependent class capabilities. Scout (grapple + flares), Engineer (platforms + sentries), Driller (tunnels), Gunner (zipline + shield). Ghost Ship Games described their inspiration as "basically combining Minecraft and Left 4 Dead." The team explicitly wanted a game where "all teammates would be on even footing regardless of experience" — a deliberate contrast to Left 4 Dead where veterans and newcomers had asymmetric impact. (Source: Ghost Ship Games founding history, Wikipedia.)

Community-driven development. GDC 2023 talk by lead designer Mikhail Akopyan discussed how early access community feedback shaped design priorities. Cosmetics were designed as a major component from launch, building an expressive community identity. The game sold 90,000 copies in its first week and needed 10,000 additional per month to sustain — community engagement was a survival mechanism, not a marketing strategy. (Source: Game Developer GDC 2023 coverage.)

Shared resupply as social resource. Resupply pods have 4 charges for 4 players. No enforced fairness. Good teams call out ammo status. This is low-bandwidth, high-trust communication about shared resources — the architectural equivalent of agents broadcasting their fuel state and trusting neighbors to self-regulate.

2.11 Left 4 Dead (Valve / Turtle Rock, 2008)

AI Director as macro orchestrator. Controls pacing at group level. "Adaptive Dramatic Pacing" adjusts emotional intensity based on survivor team metrics. Lessons from Counter-Strike: "constant, relentless action is wearing — but too many long, slow gaps and the player gets bored." (Source: AusGamers GDC 2009 coverage.)

Treat team as single player. "Throughout the design process we treated the entire survivor team as one player. The focus was on the whole survivor team as the player. Everyone knows the rules. The good guys work together. The jerks in the team that try to abandon the team, they die." (Source: Shacknews GDC 2009 coverage.)

Special infected as isolation punishers. Every special infected requires a teammate to rescue you. Mechanical enforcement that separation = death.

2.12 Divinity: Original Sin 2 (Larian Studios, 2017)

Emergent combos from physical simulation. Water + lightning = electrified surface. Oil + fire = area damage. These are physical rules that players chain cooperatively. No scripted combos. Systems designer Nick Pechenin: "We were using [the arena prototype] to prototype how combat would feel, what we were going to do with Action Points, what new elemental combinations we could come up with, and how we'd shape the roles and archetypes." The arena mode became a laboratory: "We knew it had to be deep, with a lot of systems and a lot of combinations that we'll probably never see. We had people teleporting, lava surfaces everywhere and everyone going around killing civilians left and right." (Source: Steam/PC Gamer interview with Pechenin; GDC 2019 talk by Swen Vincke on iteration.)

Real friendly fire as cooperation tax. Your fire spell hits your melee teammate. Your lightning electrifies the water your rogue is standing in. The surface system creates an emergent interference detection problem — agents must be aware of what surfaces exist, what their teammates are standing on, and what elemental combos their actions will trigger. The most common source of party wipes in co-op isn't enemy damage — it's friendly combo chains gone wrong.

Iteration as design methodology. Vincke's GDC 2019 talk was titled "The Making of Divinity: Original Sin 2" and focused on how the team "applied iteration at every level, with varying levels of success." The game won the BAFTA for Best Multiplayer Game. (Source: GDC 2019, Game Developer coverage.)

2.13 Helldivers 2 (Arrowhead, 2024)

Friendly fire as core design philosophy. Orbital strikes, turret deployments, and overpenetrating weapons all hit teammates. Arrowhead CCO Johan Pilestedt: "We always knew we wanted the juxtaposition between the fragility of humans with overpowered tools versus physically superior foes only vulnerable to heavy weapons. Feeling human is a key part to the Helldivers experience." The result: "a feeling of realism and immersion but also made for a ton of funny moments, like accidentally bombing your teammates." (Source: FandomWire interview, March 2024.)

Cooperation as core design philosophy. Pilestedt: "Our core design philosophy, when it comes to cooperative shooters, is that we want players to want to play with other players. So, if you've thought, 'It would be great if I had a friend that joined me,' that's what we're looking for." (Source: CGMagazine interview, February 2024.)

Dynamic gathering locations as cooperation infrastructure. When shifting from top-down to third-person, Arrowhead "struggled with this early on" in ensuring players still felt the need to cooperate since each player now had their own perspective. The solution: "dynamic gathering locations" or "watering holes" — shared resupply stratagems and

sample drop containers that create convergence points, forcing players to coordinate spatially. (Source: FandomWire development insights, March 2024.)

Fundamentals over features. Pilestedt at GDC: game design should follow three steps — fundamentals, style, then features. “When I talk to people that are in the business side of things, [they say] it’s the price point, it’s the galactic war...” But the real driver is fundamentals: studio values, target fantasies, core feel. The studio motto: “a game for everyone is a game for no one.” (Source: PC Gamer GDC 2025 coverage.)

Believability as design principle. Pilestedt: “Our fundamental belief is that the game experience should be believable — i.e. if you guess what the outcome should be based on real life experiences, the game should match that outcome.” Not realism per se — believability. (Source: FandomWire, January 2026.)

2.14 Overcooked (Ghost Town Games, 2016)

Cooperation as the only design pillar. Phil Duncan (Design Director): “With Overcooked we wanted to make a game where cooperation was a central pillar, a game which was much more focused on how a team works together rather than simply adding more players to a single player experience.” And: “There are lots of games which allow players to work as a team, but more-often-than-not the focus is on the skills and abilities of the individuals rather than how good the team is at communicating or coordinating their behavior.” (Source: Gamasutra Game Design Deep Dive, 2016.)

Kitchen as cooperation analogy. Duncan: “Having worked in various restaurants over the years, kitchens have always struck me as a perfect analogy for a cooperative game: an occupation where teamwork” was paramount. “When time was against you it would invariably end in lots and lots of shouting.” (Source: Game Developer Road to the IGF interview.)

Equal responsibility, no carrying. “Rather than just make a game where one player could carry the rest of the team, with Overcooked all players are equally responsible for the success of the team.” (Source: Road to the IGF.) The architectural equivalent: formations should not allow a single agent to compensate for others’ failures indefinitely.

Disruptions as defining design moment. “We quickly realized that players enjoyed having to think on their feet and respond to the changing levels.” Moving counters, earthquakes separating players, sliding kitchens on pirate ships. “Suddenly players were having to really concentrate on what every member of the team was doing and stay in constant communication... more than that they were also being forced to change their roles midway through a round.” (Source: Game Design Deep Dive.) This is It Takes Two’s dynamic capability reassignment, but continuous rather than chapter-based.

Freedom vs restriction balance. “It was a real balancing act, we were constantly adjusting

the ratio of freedom and restriction that we gave the players." (Source: Road to the IGF.) This is the orchestration/cooperation balance stated as a design problem.

Task pipeline self-organization. Players share a kitchen. No station assignment. Self-organize based on proximity, current bottleneck, and observation of partner's current action. Physical obstacles force cooperation chains and make ingredient handoffs the critical coordination point. Oli De-Vine (NYU IGF interview): "Communication is a skill you need, more than any other, to succeed at Overcooked."

Emergent communication. When online play was added for Overcooked 2, Ghost Town added an emote system. But players also "communicating in ways we weren't expecting — spinning on the spot, or throwing uncooked chickens at one another." (Source: Push Square interview, 2018.) Agents will develop their own signaling protocols if the system allows any form of observable behavior.

3. Orchestration Boundary

This section defines what the scoped `orchestrator/` module owns. Everything not listed here belongs to `cooperation/`.

```
crates/springtale-bot/src/orchestrator/
├─ mod.rs
├─ composer.rs          - select which agents form a group (pre-mission)
├─ intent.rs            - publish intent patterns to cadence bus
├─ constraints.rs        - fuel budgets, guard mode, autonomy ceiling
└─ intervention.rs      - rally on cascade failure, dissolve stuck formations
```

3.1 Composer — Pre-Mission Army Selection

Game source: Patapon army selection, Total War recruitment, Siege operator pick, Deep Rock class selection.

The composer selects which agents participate and assigns initial role hints. This happens BEFORE execution. Like Patapon's army composition screen, the composer decides what capabilities are available. What happens once they're in the field is cooperation.

```
pub struct FormationComposition {
    pub formation_id: FormationId,
    pub members: Vec<AgentSlot>,
    pub intent: IntentPattern,
    pub constraints: FormationConstraints,
}
```

```

pub struct AgentSlot {
    pub agent_id: AgentId,
    pub capabilities: Vec<CapabilityDecl>,
    pub role_hint: Option<RoleHint>,          // suggestion, not mandate
}

```

The `role_hint` is like equipping a Patapon with fire arrows — it biases behavior without mandating it.

3.2 Intent — The Drum Pattern

Game source: Patapon drum patterns, Total War attack/defend orders, Siege IGL strat calls.

Intent describes WHAT, never HOW. “Attack” tells the formation to engage. It does not tell individual agents which target to pick, what timing to use, or what sequence to follow.

```

pub enum IntentPattern {
    /// Gather information. Sensor agents activate.
    /// Patapon: PATA PATA PATA PON. Siege: Drone phase.
    Reconnoiter { target: TaskDescriptor },

    /// Execute against a known target.
    /// Patapon: PON PON PATA PON. Total War: Charge.
    Execute { plan_id: Option<PlanId> },

    /// Hold current state. Defensive agents activate.
    /// Patapon: CHAKA CHAKA PATA PON. Total War: Guard mode.
    Stabilize { reason: StabilizeReason },

    /// Maximum commitment to singular objective.
    /// Patapon: DON DON CHAKA CHAKA. Army of Two: Overkill.
    Surge { objective: TaskDescriptor },

    /// Graceful wind-down.
    Dissolve { reason: DissolveReason },
}

```

3.3 Constraints — Guard Mode Toggles

Game source: Total War guard/free mode, Siege round timer, sentinel controls.

```

pub struct FormationConstraints {
    pub fuel_budget: FuelBudget,
    pub timeout: Duration,                // Siege round timer
}

```

```

pub max_concurrent_actions: usize,
pub destructive_action_policy: ApprovalPolicy, // always L1
pub guard_mode: bool,                        // Total War: don't pursue
pub autonomy_ceiling: AutonomyLevel,
}

```

3.4 Intervention — The General's Rally

Game source: Total War general rally, Left 4 Dead Director, Patapon rhythm switch.

Reactive and exceptional. Only when cooperation self-recovery (§15) fails.

```

pub enum Intervention {
    ChangeIntent(IntentPattern), // Patapon: switch rhythm
    InjectFuel(FuelBudget),      // L4D: health kit spawn
    ForcedDissolve { reason: String },
    EscalateToUser { summary: ActionSummary },
}

```

4. Module Structure

```

crates/springtale-bot/src/
├── orchestrator/                # SCOPED: composition, intent, constraints, interv
│   ├── mod.rs
│   ├── composer.rs
│   ├── intent.rs
│   ├── constraints.rs
│   └── intervention.rs
│
├── cooperation/                # NEW: everything between intent and outcome
│   ├── mod.rs
│   ├── cadence.rs              # §5  - shared tick bus
│   ├── formation.rs            # §6  - peer agent grouping
│   ├── momentum.rs             # §7  - coherence accumulator / Fever
│   ├── awareness.rs            # §8  - local neighbor perception
│   ├── attention.rs            # §9  - workload economy / aggro
│   ├── environment.rs          # §10 - shared mutable workspace
│   ├── consensus.rs            # §11 - weighted decision resolution
│   ├── commit.rs               # §12 - synchronized execution barriers
│   ├── interference.rs         # §13 - conflict detection
│   ├── transformation.rs       # §14 - role change on capability loss
│   ├── rally.rs                # §15 - cascade recovery
│   └── capability.rs           # §16 - dynamic capability binding

```

		recovery.rs	# §18 – distress detection, mutual aid, escalating
		comms.rs	# §19 – multi-layer communication protocols
		handoff.rs	# §20 – work product transfer between agents
		mental_model.rs	# §21 – shared context accumulation
		pacing.rs	# §22 – work/rest cycle management
		sacrifice.rs	# §24 – deliberate self-cost for team benefit

5. Cadence System

Game sources: Necrodancer (external clock), Patapon (rhythm as intent), Overcooked (implicit timing).

5.1 External Clock, Not Parent-Driven

Necrodancer insight: neither player owns the beat. The music IS the clock. All participants synchronize to it. Ryan Clark's 100% timing leeway discovery means agents should have generous windows to commit actions — the hard part is choosing the RIGHT action, not hitting the timing.

```
pub struct CadenceBus {
    tick_interval: Duration,
    current_intent: RwLock<IntentPattern>,
    tick_counter: AtomicU64,
    tx: broadcast::Sender<Tick>,
}

pub struct Tick {
    pub sequence: u64,
    pub timestamp: Instant,
    pub intent: IntentPattern,
    pub window: Duration,    // generous, per Necrodancer insight
}

pub struct TickReport {
    pub agent_id: AgentId,
    pub tick_sequence: u64,
    pub action_taken: Option<ActionDescriptor>,
    pub latency: Duration,
    pub intent_alignment: f32,
    pub interference_with: Vec<AgentId>,
}
```

5.2 Intent Transitions

Like Patapon switching rhythms mid-mission. Three sources: orchestrator command, formation self-governance (via consensus at Fever tier), or momentum-gated access to new intent options.

6. Formation System

Game sources: Deep Rock (interdependent classes), Monster Hunter (emergent roles), Siege (anchor/roamer).

6.1 Peer Groups, Not Hierarchies

Deep Rock's team has no leader. Nobody assigns the Scout to grapple up. Each class's capabilities intersect with the environment to produce self-organized cooperation. Formations replace the parent→child pipeline.

```
pub struct Formation {
    pub id: FormationId,
    pub members: Vec<FormationMember>,
    pub intent: IntentPattern,
    pub constraints: FormationConstraints,
    pub momentum: MomentumState,
    pub environment: SharedEnvironment,
    pub cadence: CadenceBus,
}

pub struct FormationMember {
    pub agent_id: AgentId,
    pub capabilities: Vec<CapabilityDecl>,
    pub current_role: DynamicRole,
    pub awareness: LocalAwareness,
    pub attention_load: f32,
    pub fuel_remaining: FuelBudget,
    pub health: AgentHealth,
}
```

7. Momentum System

Game sources: Patapon Fever (qualitative unlocks), Total War veterans, Monster Hunter topple windows.

7.1 Tiers, Not Scores

Patapon Fever doesn't make units "10% stronger" — it unlocks attack patterns that don't exist outside Fever. Momentum determines what agents CAN do.

```
pub enum MomentumTier {
    /// Just assembled. Read-only environment. No chaining.
    Cold,
    /// 3+ successful ticks. Can read neighbor reports. Basic chaining.
    Warming,
    /// 8+ successful ticks, low interference. Write to environment. Complex chain
    Hot,
    /// 15+ successful ticks, zero interference. Consensus unlocked. AI adapter a
    Fever,
}
```

Capability	Cold	Warming	Hot	Fever
Read shared environment	✓	✓	✓	✓
Read neighbor TickReports	—	✓	✓	✓
Chain: A's output feeds B	—	✓	✓	✓
Write shared environment	—	—	✓	✓
Synchronized commit	—	—	✓	✓
Formation consensus	—	—	—	✓
AI adapter calls	—	—	—	✓
Recruit additional agents	—	—	—	✓

8. Awareness System

Game sources: Total War morale (composite local signal), Siege callouts, Splinter Cell split approach.

```
pub struct LocalAwareness {
    pub neighbor_states: HashMap<AgentId, NeighborSnapshot>,
    pub formation_momentum: MomentumTier,
    pub last_tick_reports: Vec<TickReport>, // Warming+ only
```

```

}

pub struct NeighborSnapshot {
    pub agent_id: AgentId,
    pub health: AgentHealth,
    pub current_role: DynamicRole,
    pub fuel_remaining_pct: f32,
    pub last_action_success: bool,
    pub attention_load: f32,
    pub last_updated: Instant,
}

```

9. Attention Economy

Game source: Army of Two aggro system.

One agent's high workload consumption directly enables another's freedom. Zero-sum within the formation.

```

pub struct AttentionEconomy {
    pub total_attention: f32,
    pub distribution: HashMap<AgentId, f32>, // sums to 1.0
}

```

Like Army of Two's Overkill mode, extreme concentration (>90% on one agent) could trigger a time-limited power state.

10. Shared Environment

Game sources: Siege destructible walls, Divinity surface system.

```

pub struct SharedEnvironment {
    pub workspace: HashMap<String, serde_json::Value>,
    pub write_log: Vec<EnvironmentWrite>,
    pub surfaces: Vec<Surface>,
}

pub struct Surface {
    pub created_by: AgentId,
    pub surface_type: SurfaceType,
    pub data: serde_json::Value,
}

```

```

    pub expires: Option<Instant>,
}

pub enum SurfaceType {
    Substrate, // Divinity: water on ground
    Primed { trigger: ActionDescriptor }, // Divinity: oil ready to ignite
    Active { remaining: Duration }, // Divinity: fire burning
}

```

11. Consensus Engine

Game source: As Dusk Falls voting + override system.

```

pub struct ConsensusVote {
    pub question: DecisionDescriptor,
    pub votes: HashMap<AgentId, VoteChoice>,
    pub deadline: Instant,
    pub overrides_remaining: HashMap<AgentId, u32>,
}

pub enum VoteResolution {
    Majority(VoteChoice),
    Override { by: AgentId, choice: VoteChoice, cost: u32 },
    Timeout(VoteChoice),
}

```

Overrides are visible to all members and cost a scarce resource. Only at Fever tier.

12. Synchronized Commit

Game sources: Splinter Cell dual breach, Army of Two co-op snipe.

```

pub enum CommitPhase {
    Prepare,
    Ready,
    Countdown { remaining: Duration },
    Execute,
    Collect,
}

```

Available at Hot+ tier. Cooperation is in planning; execution is deterministic.

13. Interference Detection

Game sources: Helldivers 2 friendly fire, Divinity combos hitting allies, Total War archers hitting own infantry.

```
pub struct InterferenceEvent {
    pub tick_sequence: u64,
    pub agent_a: AgentId,
    pub agent_b: AgentId,
    pub interference_type: InterferenceType,
    pub severity: f32,
}

pub enum InterferenceType {
    ResourceConflict,    // both modified same resource
    ActionNegation,      // A undid B's work
    CollateralDamage,    // A's side effects harmed B
    Redundancy,          // both did the same thing
}
```

Interference decreases momentum. Repeated interference triggers role adaptation.

14. Role Transformation

Game sources: Siege dead→intel, Army of Two role oscillation, It Takes Two chapter abilities.

```
pub enum RoleTransformation {
    /// Primary capability lost. Becomes information-only. Siege: dead→cameras.
    ToInformationAgent,
    /// Primary exhausted. Becomes support. Army of Two: low-aggro→overwatch.
    ToSupportAgent,
    /// Context changed. New tools. It Takes Two: new chapter.
    ReassignCapabilities(Vec<CapabilityDecl>),
}
```

15. Rally & Cascade Recovery

Game sources: Total War general rally, routing cascade, Monster Hunter carts.

15.1 Cascade Detection

Like Total War's routing contagion: Agent A fails → neighbors see it → their health drops → cascade risk.

15.2 Formation Self-Rally (before escalating to orchestrator)

1. Redistribute attention (§9) away from struggling agent
2. Transform roles (§14) for failed agent
3. Reduce momentum tier to match reduced coherence
4. Consume rally token (limited, like Monster Hunter carts)

15.3 Escalation

Only if self-rally fails (tokens consumed, Cold momentum, multiple agents failing) does the formation escalate to `orchestrator::intervention`.

16. Dynamic Capability Binding

Game source: It Takes Two chapter-based reassignment.

```
pub struct DynamicCapabilitySet {  
    pub base_capabilities: Vec<CapabilityDecl>,          // connector manifest  
    pub context_capabilities: Vec<CapabilityDecl>,       // formation context  
    pub momentum_unlocked: Vec<CapabilityDecl>,         // momentum tier  
    pub transformed_capabilities: Vec<CapabilityDecl>,  // role transformation  
}
```

17. Integration with Existing Architecture

17.1 Sentinel — Unchanged

Monitors both orchestrator and cooperation layers. Rate limiters, circuit breakers, dead-man switch, audit trail apply regardless of source.

17.2 Pipeline Engine — Recontextualized

`compose_pipeline()` still works. Pipelines become playbooks that formations execute together, rather than trees a parent imposes on children.

17.3 Connector Sandbox — Unchanged

WASM sandbox, manifest signing, capability enforcement remain.

17.4 Autonomy Levels — Extended

L1–L4 still apply. Momentum tiers interact: an L2 agent in a Fever formation can participate in consensus but still requires plan approval. Destructive actions remain L1 regardless.

17.5 Migration Path

Phase 2a: `cooperation/` ships alongside existing orchestrator/. Legacy pipelines w

Phase 2b: Legacy users migrate to formations. `recursive.rs` deprecated.

Phase 3: `recursive.rs` and `subagent.rs` removed. Formation-only coordination.

18. Recovery & Mutual Aid System

The other side of cooperation is agents actively seeking out and helping each other.

Every cooperative game listed in §2 has a recovery system, and no two are identical. The variation in approach reveals distinct architectural patterns for how agents should detect distress, deliver aid, and manage the cost of helping.

This section adds `recovery.rs` to the `cooperation/` module.

18.1 Recovery Patterns Across All Games

Left 4 Dead — The deepest recovery system in cooperative games.

L4D has four distinct recovery mechanisms layered on top of each other:

Incapacitation and peer revive. When health hits zero, the survivor falls with 300 temporary HP that bleeds at 3/tick. A teammate must reach them and hold interact for 5 seconds — during which BOTH the helper and the downed player are vulnerable. The downed player has only a pistol for defense. After revive: 1 permanent HP + 29 temporary HP. This is a degraded state, not full recovery.

Escalating fragility. After two incapacitations without a medkit heal, the player enters “black and white” mode — grayscale vision, heartbeat audio, blurred edges. The next incapacitation is death. This means each recovery without proper healing makes the NEXT failure more catastrophic. The system punishes repeated band-aid fixes and rewards investing in proper restoration.

Heal-other mechanic. First Aid Kits can be used on yourself OR on a teammate. Using a kit

on a teammate means you spend YOUR scarce resource on THEM. It takes 5 seconds, during which you can't defend yourself. The kit heals 80% of missing permanent health and resets the incapacitation counter. Pain pills give 50 temporary HP that bleeds off — a quick fix, not a real heal.

Dead player recovery. Defibrillators (L4D2) revive dead teammates, not just downed ones. Dead players also respawn in rescue closets at checkpoints with 50 HP and downgraded weapons. The game provides multiple paths back from failure, each with different costs and quality of recovery.

Architectural takeaway: Recovery has TIERS. Quick-fix recovery (pills/revive) is cheap but leaves the agent in a degraded state. Proper recovery (medkit) is expensive but restores full capability. Repeated quick-fixes without proper recovery creates escalating fragility. Dead agents can still be recovered but at higher cost and with capability loss.

Deep Rock Galactic — Environmental healing + peer revive + conditional self-revive.

Peer revive. Hold interact for 6 seconds near a downed dwarf. The Field Medic perk allows one instant revive per mission — a scarce emergency resource.

Environmental healing. Red Sugar crystals are scattered through caves. They heal anyone who mines them. They're a fixed, non-renewable environmental resource — first come, first served. Teams must decide who needs it most.

Shared resource healing. Resupply pods provide ammo AND health. Each pod has 4 charges for 4 players. Taking a charge when you don't need ammo just for health means one less charge for a teammate who needs both.

Conditional self-revive (Iron Will). The Iron Will perk gives 10 seconds of temporary revival. But you're at 0 HP — if the timer runs out without getting real health (Red Sugar, resupply, Vampire perk kills), you go back down. The strategic depth: use those 10 seconds to revive a teammate (who can then revive you properly), reach a resupply pod, or find Red Sugar. You're "planning for failure" by building recovery into your loadout.

Gunner's shield as recovery infrastructure. The Gunner's deployable shield creates a safe zone. This enables revives under fire — one player deploys the shield, then revives the downed player inside it. The shield is a tool-assisted recovery, not a healing item.

Architectural takeaway: Recovery resources come from multiple sources — environmental (fixed, shared, non-renewable), shared consumable (pods with limited charges), peer action (revive), and conditional self-recovery (Iron Will requiring additional resource acquisition to stabilize). The Gunner's shield pattern shows that agents can CREATE recovery conditions for others using their capabilities, even if they don't have a "heal" ability.

Monster Hunter — Support as optional specialization, not designated role.

Self-healing is primary. Every hunter carries potions. The base expectation is that you heal yourself. Drinking a potion has an animation that leaves you vulnerable — a cost that YOU accept for YOUR recovery.

Wide-Range skill. An armor skill that shares item effects with nearby teammates. When you drink a Mega Potion with Wide-Range 5, all nearby allies heal too. This turns self-healing into team-healing as a side effect, not a dedicated action. The experienced community is split on whether Wide-Range support builds are valuable or a crutch. Key debate: a dedicated support player who heals but doesn't attack is criticized ("don't be a backseat dooter"), while a player who attacks normally and ALSO heals via Wide-Range is celebrated. The lesson: healing should be a BYPRODUCT of normal operation, not a replacement for it.

Hunting Horn buffs as proactive healing. The Hunting Horn applies buffs (attack up, health recovery, earplugs, wind resistance) to all nearby allies through combat melodies. The Horn's healing is embedded in its attack combos — you heal by fighting, not by stopping to heal. This is the most architecturally relevant pattern: recovery as a side effect of productive work, not a separate activity.

Lifepowder / Dust of Life. Instant team heal items. Limited supply. The Monster Hunter community calls using Dust of Life "the biggest flex" because it signals you're paying attention to teammates' health while fighting the monster yourself.

Architectural takeaway: The ideal recovery model has healing as a byproduct of productive work, not a dedicated role. Agents that naturally produce recovery effects for neighbors while doing their primary job are more valuable than agents that stop their primary work to heal. The hunting horn pattern — attacking IS healing — is the design goal.

Army of Two — Recovery as cooperative action requiring physical commitment.

Drag-and-heal. When a player goes down, the partner must physically reach them under fire, drag them to cover (both are vulnerable during the drag), and then heal. The healthy player faces a forced choice: press the tactical advantage (while the partner bleeds out) or expose themselves to save the partner.

Aggro manipulation for recovery. The surviving player can draw maximum aggro to give the downed player's AI partner space to revive. Recovery is enabled by the attention economy — one agent consuming all the threat so another can perform the vulnerable recovery action.

Architectural takeaway: Recovery often requires one agent to sacrifice their current operational advantage (stop attacking, expose position, consume attention) to help another. This cost must be explicit and measured. The aggro system lets agents create safe windows for recovery.

Rainbow Six Siege — Specialized recovery operators + universal DBNO.

DBNO (Down But Not Out) state. Downed operators can be revived by any teammate. Rook's armor plates guarantee DBNO instead of instant death — proactive protection that changes the failure mode from death to recoverable state.

Doc operator. Can self-revive from DBNO (unique). Can shoot healing darts at teammates from range — recovery at a distance. Has limited stim charges (3 per round). The healer must triage: who gets the limited charges.

Finka operator. Global team boost — activates a temporary health surge for ALL teammates simultaneously, regardless of position. Can revive DBNO teammates remotely. This is a formation-wide recovery pulse, not targeted healing.

Dead player as persistent asset. Dead operators switch to cameras and provide callouts. They're not "recovered" — they're transformed (§14). But their continued information contribution IS a form of the team recovering from their loss.

Architectural takeaway: Recovery can be targeted (Doc's darts) or formation-wide (Finka's boost). Proactive protection (Rook's armor) can change failure modes from unrecoverable to recoverable. Some agents should specialize in recovery as a secondary capability, not a primary role. Dead agents that transform to intel are a form of recovery — the formation recovers capability even though the individual doesn't.

Helldivers 2 — Death as expected, redeployment as recovery.

Reinforce stratagem. Dead teammates are recovered by calling in a reinforce stratagem — they literally drop back in from orbit. Death is not a failure state to avoid at all costs; it's an expected event with a built-in recovery mechanism. The cost: calling the stratagem takes time, uses a shared cooldown, and the redeployed player loses their loadout/stratagems.

Stim self-heal and teammate stim. Players can stim themselves or inject nearby teammates. The heal is immediate but limited by stim count.

Resupply as shared recovery point. Pilestedt described creating "dynamic gathering locations" through shared resupply stratagems. These convergence points serve as recovery infrastructure — agents come together, resupply, heal, and coordinate before spreading out again.

Architectural takeaway: If agent failure is expected and frequent (like in Helldivers), the recovery system should treat replacement/redeployment as a first-class operation, not an emergency measure. Resupply convergence points are recovery infrastructure that also enables coordination — recovery and cooperation reinforce each other.

Total War — The general as a physical recovery agent.

Rally mechanic. The general's rally ability restores morale to routing units within the

general's influence radius. But units whose morale is "shattered" will not respond to rally. The general is a physical entity that must be POSITIONED correctly to provide recovery — you can't rally from across the battlefield.

Resting recovers fatigue. Units pulled out of combat gradually recover stamina. Recovery requires disengagement — you can't recover while still engaged.

Reinforcement armies. Fresh armies can arrive mid-battle from off-map. This is formation-level recovery — injecting new capability when existing agents are depleted.

Architectural takeaway: Recovery has a physical/proximity component (the rallying agent must be near the failing agent). Recovery has limits (shattered morale can't be rallied). Recovery requires disengagement (fatigued units must stop fighting to recover). New agents can be injected as a recovery mechanism.

Splinter Cell — 30-second revive window with shared consequences.

When a player is downed, the other has 30 seconds to reach and revive them. Mission fails if both are down simultaneously. The revive timer creates urgency. The shared failure state (mission over if both down) means the surviving player must balance their own safety against the revive attempt.

Patapon — Proactive protection over reactive healing.

The defend rhythm (CHAKA CHAKA PATA PON) raises shields across the army. This is formation-wide damage prevention, not healing. Recovery between missions: fallen Patapons can be revived using resources collected during the mission. The game emphasizes preventing damage through rhythm-appropriate behavior over recovering from it.

Architectural takeaway: Sometimes the best recovery is prevention. A well-timed defensive intent switch (Stabilize) can prevent the need for healing entirely.

Crypt of the Necrodancer — No peer healing. Prevention is everything.

In standard co-op, each player manages their own health from environmental pickups (food items on the dungeon floor). There is no player-to-player healing. In shared health pool mode, every hit to either player costs the team. Recovery means finding food items on the grid. The cooperation challenge is preventing damage through spatial deconfliction, not healing after it occurs.

Divinity: Original Sin 2 — Healing as resource competition with combat.

Healing spells cost Action Points (AP). AP spent healing is AP not spent attacking. Resurrection scrolls are rare and expensive. The healing economy creates a direct tension: investing in recovery means less offensive capability. Necromancy inverts this — dealing

damage IS healing (lifesteal). The surface system also creates unintentional “anti-healing” when friendly fire damages allies.

Architectural takeaway: Recovery should cost the same resource as productive work (fuel), creating a real trade-off. Agents that find ways to recover as a byproduct of productive work (like Necromancy’s lifesteal or Hunting Horn’s combat melodies) break this trade-off in valuable ways.

It Takes Two — Generous checkpoints as no-cost recovery.

No health system in most sections. Death means instant respawn at a nearby checkpoint. The game deliberately removes recovery cost to keep cooperation flowing — failure is expected during the learning curve of each chapter’s new mechanics. The “recovery” is: try again immediately, with no penalty.

Architectural takeaway: For formation types where experimentation and learning are the goal (prototyping, exploration), recovery should be nearly free. The cost of failure should be low enough that agents try novel approaches.

As Dusk Falls — Override as recovery from bad group decisions.

No mechanical healing. The override token IS the recovery mechanism — it lets a player “heal” a bad group decision by forcing a different outcome. This costs a scarce resource and is visible to all. Narrative consequences from earlier bad decisions can sometimes be mitigated by later good decisions.

Overcooked — Error correction as recovery.

“Recovery” means fixing mistakes in the pipeline. Burned dishes must be washed before the station can be reused. Fires must be extinguished. Dropped ingredients must be re-prepared. Recovery costs TIME — the formation’s shared temporal budget. Phil Duncan’s “disruptions” (earthquakes, moving kitchens) create recovery challenges: when the kitchen layout changes, previously prepared ingredients may become stranded and the team must adapt.

Architectural takeaway: In pipeline-based formations, recovery means error correction and re-work. The cost is temporal, not resource-based. Prevention of errors (better coordination) is cheaper than recovery from errors.

18.2 Recovery Architecture

```
// crates/springtale-bot/src/cooperation/recovery.rs
```

```
/// How an agent signals it needs help
pub enum DistressSignal {
```

```

    /// Agent health below threshold. Total War: morale dropping.
    HealthLow { agent_id: AgentId, health_pct: f32 },
    /// Agent incapacitated. L4D: downed state. DRG: downed dwarf.
    Incapacitated { agent_id: AgentId, bleedout_remaining: Duration },
    /// Agent dead/disconnected. Helldivers: needs reinforce.
    Dead { agent_id: AgentId, recoverable: bool },
    /// Agent capability degraded. Siege: DBNO with limited actions.
    Degraded { agent_id: AgentId, remaining_capabilities: Vec<CapabilityDecl> },
}

/// How recovery is delivered
pub enum RecoveryAction {
    /// Peer revive – one agent directly restores another.
    /// Cost: the healer is vulnerable during the action.
    /// L4D medkit, DRG revive, Splinter Cell revive, Army of Two drag-heal.
    PeerRevive {
        healer: AgentId,
        target: AgentId,
        duration: Duration,
        healer_vulnerability: f32, // 0.0-1.0: how exposed the healer is
    },

    /// Byproduct recovery – agent heals neighbors by doing its normal work.
    /// MH Hunting Horn: attack combos apply healing melodies.
    /// MH Wide-Range: self-healing shares to team.
    /// Divinity Necromancy: dealing damage heals self.
    ByproductRecovery {
        source: AgentId,
        beneficiaries: Vec<AgentId>,
        recovery_amount: f32,
        primary_action: ActionDescriptor, // the productive work that caused hea
    },

    /// Formation-wide pulse – all agents get a boost simultaneously.
    /// Siege Finka boost. Helldivers reinforce. Patapon defend rhythm.
    FormationPulse {
        source: AgentId,
        recovery_amount: f32,
        cost: RecoveryCost,
    },

    /// Environmental recovery – agent uses a shared resource.
    /// DRG Red Sugar. L4D safe room. Helldivers resupply convergence.
    EnvironmentalRecovery {
        source_resource: ResourceId,
        beneficiary: AgentId,
        depletes_resource: bool,
    },
}

```

```

    },

    /// Redeployment – replace a dead agent entirely.
    /// Helldivers reinforce. L4D rescue closet.
    Redeployment {
        dead_agent: AgentId,
        replacement_capabilities: Vec<CapabilityDecl>,
        cost: RecoveryCost,
        degraded: bool, // L4D rescue closet: Tier 1 weapons only
    },

    /// Proactive protection – change formation posture to prevent damage.
    /// Patapon defend rhythm. DRG Gunner shield. Rook armor plates.
    ProactiveProtection {
        protector: AgentId,
        beneficiaries: Vec<AgentId>,
        protection_type: ProtectionType,
    },
}

pub enum RecoveryCost {
    /// Recovery costs fuel from the healer's budget.
    Fuel(FuelBudget),
    /// Recovery costs shared formation fuel.
    SharedFuel(FuelBudget),
    /// Recovery costs time (Overcooked error correction).
    Time(Duration),
    /// Recovery costs a scarce token (As Dusk Falls override, DRG Iron Will).
    Token { token_type: String, remaining_after: u32 },
    /// Recovery is free (It Takes Two checkpoint).
    Free,
}

pub enum ProtectionType {
    /// Shield/barrier that blocks damage. DRG Gunner shield.
    DamageShield { duration: Duration },
    /// Posture change that reduces incoming damage. Patapon defend.
    PostureChange { damage_reduction: f32 },
    /// Preemptive buff that changes failure mode. Rook armor = DBNO not death.
    FailureModeChange { from: FailureMode, to: FailureMode },
}

pub enum FailureMode {
    /// Agent goes from operational to dead. No recovery window.
    InstantDeath,
    /// Agent enters degraded state with recovery window. L4D incapacitation.
    Degraded { recovery_window: Duration },
}

```

```

    /// Agent transforms role instead of failing. Siege dead→intel.
    RoleTransformation,
}

```

18.3 Recovery Tiers (L4D-Inspired Escalating Fragility)

Following Left 4 Dead’s escalating fragility model, recovery quality degrades with repeated use:

Recovery Count	State	Capability	Next Failure
0 (healthy)	Full operational	All capabilities	→ Tier 1 recovery
1 (quick-fixed)	Degraded (L4D: 30 HP temp)	Most capabilities, reduced resources	→ Tier 2 recovery
2 (quick-fixed twice)	Critical (L4D: black & white)	Limited capabilities, visual degradation	→ Dead/removed
Proper recovery	Restored	Full capabilities, counter reset	→ Tier 1 recovery

Quick-fix recovery (peer revive, byproduct healing) increments the counter. Proper recovery (expensive full restore, environmental recovery at convergence point) resets it. This prevents formations from indefinitely limping along on band-aid fixes.

18.4 Recovery Decision Framework

When a distress signal is detected through the awareness system (§8), neighboring agents evaluate:

1. **Can I help?** Do I have the capability/resource to deliver recovery?
2. **Should I help?** Is the cost to me (vulnerability, fuel, time) justified by the value of recovering the distressed agent? (Army of Two’s press-attack-vs-save-partner dilemma)
3. **Should someone else help?** Is another agent better positioned or less costly to deploy? (Monster Hunter: the player nearest the downed hunter revives, not the one dealing the most damage)
4. **Should we prevent instead?** Would switching to a defensive posture (Patapon defend, DRG Gunner shield) prevent more damage than reactive healing would fix?

5. **Should we let them transform?** Is the distressed agent more valuable in a transformed role (Siege dead→intel) than in a recovered-but-degraded state?

This evaluation happens locally per agent through the awareness system, NOT centrally through the orchestrator. The orchestrator only sees recovery through momentum changes — if the formation self-heals successfully, momentum stabilizes. If it doesn't, momentum drops and eventually triggers orchestrator intervention (§3.4).

18.5 Recovery and Rekindle's Mutual Aid Principle

This maps directly to Rekindle's Design Principle 12: "Mutual aid is the incentive. No tokens. No payments. No reward systems. The reward is the network getting better for you. Relay for your friend → your messages deliver faster. Cache community emoji → the picker loads instantly for everyone. Pin a file → the whole community has it. Leave a rope. Someone grabs it. Later they leave a ladder. You climb it."

Every recovery action in the cooperation layer follows this principle. An agent that helps recover a neighbor benefits from having that neighbor operational again. The Hunting Horn pattern is the purest expression: you heal by fighting, and a healed team fights better, which makes your fighting more effective. Recovery is not charity — it's mutual aid that improves the system for everyone, including the helper.

18.6 Module Addition

```
crates/springtale-bot/src/cooperation/  
├─ ...existing modules...  
└─ recovery.rs      # §18 — Distress detection, recovery delivery, escalating fra
```

19. Communication Protocols

Every cooperative game solves inter-agent communication differently. The variation reveals that agents need multiple simultaneous communication layers, not a single channel.

19.1 Communication Patterns Across All Games

Left 4 Dead — Automatic state broadcasts. Valve recorded over 1,000 unique voice lines per survivor. These fire AUTOMATICALLY based on game state — the player doesn't press a button to call out a Witch. When a survivor spots a special infected, takes damage, sees a teammate go down, or finds supplies, the character vocalizes it. This is state-triggered broadcast communication. Additionally, weapon-mounted flashlights and muzzle flashes

communicate player actions visually — “lights help players determine when their companions are shooting, performing melee attacks, reloading or moving.” (Source: Wikipedia.)

Siege — Emergent vocabulary. The community created a dense spatial vocabulary (room names, position callouts, shorthand) because the game shipped without one. “Jäger’s in trophy,” “rotate to site,” “they’re pushing garage.” The game has a contextual ping system, but the emergent voice callouts are far richer. Dead players become pure information broadcasters — switching to cameras and providing positional intelligence they couldn’t gather while alive.

Monster Hunter World — Cross-context translation. MH shipped with a pre-translated greeting/command system because the game supports cross-region cooperative play. Simple messages like “I need healing” are automatically translated. This is a designed low-bandwidth protocol for when agents don’t share a common representation. (Source: Wikipedia MHW multiplayer design.)

Overcooked — Emergent signaling through observable behavior. Duncan described players “communicating in ways we weren’t expecting — spinning on the spot, or throwing uncooked chickens at one another.” (Source: Push Square interview, 2018.) When formal communication channels are insufficient, agents invent signals from any observable behavior available. Oli De-Vine: “Communication is a skill you need, more than any other, to succeed at Overcooked.” (Source: NYU IGF interview, 2017.)

Patapon — Bidirectional confirmation. The player drums. The Patapons sing back, chanting the rhythm. Kotani described this as “not just a simple game: it’s like creating the music with the Patapons.” (Source: Gamasutra interview.) The agents confirm receipt of intent through observable behavior — the commander knows the agents received the order because they can hear the response.

Deep Rock Galactic — Social cohesion signal. “Rock and Stone” is a single-button team salute. It communicates nothing tactical — it’s pure social acknowledgment. “I see you. We’re a team.” It has zero information content but maintains formation morale. Players also use the laser pointer to mark objectives, minerals, and threats — a focused attention-directing signal.

Splinter Cell — Asymmetric information fusion. Split-approach missions give each player different sensor capabilities. The high player sees through windows; the ground player hears conversations. Neither has complete information. Patrick Redding’s GDC 2011 talk distinguished between “player cooperation” (humans communicating) and “systemic cooperation” (game systems enabling coordination). (Source: Game Developer GDC 2011 coverage.) The mark-and-execute system is structured information sharing — each player marks what they can see, building a shared targeting picture.

Army of Two — The aggro meter as communication. The aggro meter is a visible gauge showing both players the current threat distribution. This is a shared dashboard — both agents can see the cooperative state at a glance. No verbal communication needed to know who has aggro.

Helldivers 2 — Stratagem callouts as spatial warnings. Every stratagem call-in is a team-wide communication event because orbital strikes kill teammates. “Orbital incoming on the left!” The stratagem input sequence itself is observable — teammates can see you punching in a code and should react. Pilestedt’s “dynamic gathering locations” through resupply stratagems also serve as implicit spatial coordination signals — “come here, there are resources.”

As Dusk Falls — Vote visibility as communication. After each decision, all players see who voted for what. The voting pattern communicates values and priorities without explicit discussion. Override usage communicates conviction strength — “I care about this more than you do.”

It Takes Two — Split screen as shared awareness. The CHI 2024 cooperative games framework noted that It Takes Two’s split screen “allows access to the other player’s perspective, which allows for better coordination, cooperation, communication and support.” Each player can see what their partner sees, enabling anticipatory coordination without verbal communication.

19.2 Communication Architecture

```
// crates/springtale-bot/src/cooperation/comms.rs

/// Multi-layer communication between formation members
pub enum CommChannel {
    /// Automatic state broadcasts. L4D survivor callouts.
    /// Triggered by conditions, not by agent decision.
    /// Low cost, high frequency, local scope.
    StateBroadcast {
        source: AgentId,
        condition: BroadcastTrigger,
        message: StateMessage,
    },

    /// Structured protocol messages. MH translated commands.
    /// Designed format, cross-context compatible.
    /// Medium cost, medium frequency.
    ProtocolMessage {
        source: AgentId,
        target: MessageTarget,
```

```

        message: ProtocolMessage,
    },

    /// Attention-directing signal. DRG laser pointer, Siege ping.
    /// Points other agents at something specific.
    DirectionalSignal {
        source: AgentId,
        target_object: ObjectReference,
        urgency: f32,
    },

    /// Social cohesion signal. DRG Rock and Stone.
    /// No information content. Maintains formation morale.
    CohesionSignal {
        source: AgentId,
    },

    /// Intent confirmation. Patapon sing-back.
    /// Agent acknowledges receipt of cadence intent.
    IntentAcknowledgment {
        source: AgentId,
        intent_confirmed: IntentPattern,
        interpretation: ActionDescriptor, // how the agent will respond
    },

    /// Observable behavior as implicit signal. Overcooked chicken-throwing.
    /// Inferred from agent actions, not explicitly sent.
    ImplicitSignal {
        source: AgentId,
        observed_action: ActionDescriptor,
        inferred_meaning: Option<InferredIntent>,
    },
}

pub enum BroadcastTrigger {
    HealthBelowThreshold(f32), // L4D: "I'm hurt pretty bad"
    ThreatDetected(ThreatDescriptor), // L4D: "WITCH!"
    ResourceFound(ResourceDescriptor), // L4D: "Pills here!"
    AgentDown(AgentId), // L4D: "Man down!"
    CapabilityExhausted(CapabilityDecl), // "I'm out of ammo"
}

pub enum MessageTarget {
    Formation, // all members
    Specific(AgentId), // one agent
    NearestCapable(CapabilityDecl), // whoever can help
}

```

19.3 Design Principles

The Overcooked insight — agents will invent communication from any observable behavior — means the system should make agent actions observable to neighbors by default. The Patapon insight — agents should confirm receipt of intent — means the IntentAcknowledgment channel feeds back into the cadence system, letting the formation know that agents are synchronized. The Siege insight — dead agents become pure information agents — ties into role transformation (§14). The MH insight — cross-context translation for heterogeneous agents — means protocol messages need a common schema even when agents have different internal representations.

20. Handoff & Transition

Work products must pass between agents. The handoff point is where most cooperative failures occur.

20.1 Handoff Patterns Across All Games

Overcooked — Handoff as the critical coordination point. Duncan's Game Design Deep Dive identified that the critical challenge isn't any individual station (chopping, cooking, plating) — it's the TRANSFER between stations. Passing ingredients across counters, between players, through serving windows. His "disruptions" (moving kitchens, earthquakes) make handoffs harder by moving the handoff points themselves. The game is fundamentally about pipeline handoff management under time pressure. (Source: Gamasutra Game Design Deep Dive.)

Splinter Cell — Sequential dependency handoffs. Player A boosts Player B to a ledge. B is now somewhere A can't reach. B must find a way to open a path back to A. The handoff is ASYMMETRIC — what A gives B (elevation) is different from what B returns (an opened door). Neither agent is permanently the giver or receiver — they trade the enabling role. (Source: GameFAQs co-op walkthrough.)

It Takes Two — Transformative handoffs. Player A shoots sap (raw material). Player B ignites it (transformation). The output (explosion, path clearance) is a third thing that neither input was independently. The handoff isn't just passing data — it's passing material that the receiver transforms into something new. Every chapter creates new transformation chains.

Deep Rock Galactic — Flexible intermediate handoffs. Scout grapples to a mineral vein and mines it. Minerals fall to the ground. ANY player picks them up. ANY player deposits them in Molly. The handoff chain has flexible intermediate steps — it doesn't matter WHO

performs each step, just that someone does.

Monster Hunter — Handoff through the shared target. Player A topples the monster (creating an opening). Player B exploits the opening (dealing damage to a vulnerable part). The “handoff” happens through the shared environment (the monster’s state), not through direct transfer between players. Neither player passes anything to the other — they both act on a shared object whose state one player changed for the other’s benefit.

Divinity OS2 — Surface handoffs. Player A creates a water surface. Player B electrifies it. The handoff medium is the shared environment — surfaces are the intermediate state. The primed surface (§10) sits in the environment waiting for the second agent’s action. The handoff is asynchronous and location-based, not direct.

Siege — Information handoffs. Drone player gathers intel. Assault player acts on it. The handoff is INFORMATION, not a physical object. Dead players hand off intelligence continuously through camera callouts. The quality of the handoff depends on the vocabulary precision (room names, position callouts).

20.2 Handoff Architecture

```
// crates/springtale-bot/src/cooperation/handoff.rs

pub enum HandoffType {
    /// Direct transfer. Overcooked: pass ingredient across counter.
    /// Synchronous — both agents must be ready.
    Direct {
        sender: AgentId,
        receiver: AgentId,
        payload: HandoffPayload,
    },

    /// Environment-mediated. Divinity surfaces, MH monster state.
    /// Asynchronous — sender deposits, receiver collects when ready.
    EnvironmentMediated {
        depositor: AgentId,
        deposit_location: EnvironmentKey,
        payload: HandoffPayload,
        transform_required: Option<TransformDescriptor>,
    },

    /// Flexible chain. DRG minerals.
    /// Any capable agent can perform the next step.
    FlexibleChain {
        originator: AgentId,
        current_step: usize,
```

```

        total_steps: usize,
        payload: HandoffPayload,
        next_capability_required: CapabilityDecl,
    },

    /// Sequential dependency. Splinter Cell boost.
    /// Agent A enables B, then B must enable A.
    SequentialDependency {
        enabler: AgentId,
        enabled: AgentId,
        return_obligation: ActionDescriptor,
    },

    /// Information handoff. Siege callouts.
    /// No physical payload — knowledge transfer.
    InformationTransfer {
        source: AgentId,
        recipients: Vec<AgentId>,
        intelligence: IntelligencePayload,
        perishable: bool, // does this info expire quickly?
    },
}

pub struct HandoffPayload {
    pub data: serde_json::Value,
    pub schema: String,
    pub produced_by: ActionDescriptor,
    pub consumable_by: Vec<CapabilityDecl>,
    pub expires: Option<Instant>,
}

```

20.3 Design Principles

The Overcooked lesson: handoff points are where cooperative failures concentrate. The system should track handoff success/failure rates and surface them through the awareness system (§8). The Divinity/MH lesson: environment-mediated handoffs decouple sender and receiver timing — the sender doesn't need to wait for the receiver. The DRG lesson: flexible chains are more resilient than fixed assignments because any capable agent can pick up a dropped handoff. The Splinter Cell lesson: sequential dependencies create mutual obligation — if A enables B, B has a responsibility to enable A in return.

21. Shared Mental Model

Teams that share a mental model cooperate with less communication overhead. The model must be built, not assumed.

21.1 Mental Model Patterns Across All Games

Research on cooperative game design established that “when mental models are shared, players are able to cooperate more effectively, because their mental models predict things in the same way.” Game mechanics must be consistent and provide predictability to enable mental model formation. (Source: Touns et al., ACM SIGGRAPH 2009, citing Cannon-Bowers et al. 1993.)

Left 4 Dead — Cultural preload. Booth: “Everyone has seen at least one zombie movie. Everyone knows the rules.” (Source: Shacknews GDC 2009.) Valve didn’t need to teach players that splitting up is dangerous — the zombie genre already taught them. The shared mental model was IMPORTED from cultural knowledge, not built in-game. This dramatically reduced the learning curve for cooperation.

Overcooked — Real-world knowledge as model. Duncan chose cooking because “everyone knows the basics of cooking, so we didn’t need to provide a tutorial.” (Source: Road to the IGF.) De-Vine: “Everyone eats! There are so many small tasks.” (Source: NYU IGF interview.) The recipes ARE the shared mental model. A burger needs a patty, bun, and lettuce — no instruction required.

Siege — Accumulated map knowledge. Teams develop shared spatial knowledge of every room, angle, and rotation on every map. The community-created callout vocabulary is an externalized shared mental model — it only works if both players know that “trophy” means the trophy room on Chalet. New players lacking this vocabulary are visibly less effective even with identical mechanical skill.

Monster Hunter — Pattern recognition. Experienced teams share knowledge of each monster’s attack patterns, openings, and vulnerability windows. When Rathalos does its signature fire breath, every veteran knows there’s a punish window on the tail. This knowledge was never explicitly communicated — it was built through repeated shared experience and community knowledge sharing.

Patapon — Rhythm as universal model. The 4-beat measure and the drum patterns ARE the shared mental model. Both the player and the Patapons operate within the same musical framework. The model is so simple that it can be learned in minutes but supports depth through the interaction of rhythm × class × equipment × environment.

Total War — Doctrinal knowledge. The 220 rules in the combat AI were derived from Sun Tzu’s Art of War. Players who have read Sun Tzu understand WHY the AI behaves as it does — the shared mental model extends outside the game into military doctrine. “When you surround an army, leave an outlet free.” Players who understand this rule position their

armies accordingly.

Deep Rock Galactic — Class capability awareness. Experienced teams have a shared model of what each class CAN do. When a Scout sees minerals on a ceiling, they don't need to ask the Engineer for a platform — both players know the Engineer platforms + Scout grapples combo. The mental model is: "I know what you can do, you know what I can do, so we can coordinate without talking."

21.2 Mental Model Architecture

```
// crates/springtale-bot/src/cooperation/mental_model.rs

pub struct SharedMentalModel {
    /// What the formation has learned about its task domain.
    /// Grows over time. MH: monster patterns. Siege: map knowledge.
    pub domain_knowledge: HashMap<String, DomainEntry>,

    /// What each agent knows about other agents' capabilities.
    /// DRG: "Engineer has platforms." Siege: "Thermite has hard breach."
    pub capability_awareness: HashMap<AgentId, Vec<CapabilityDecl>>,

    /// Successful patterns the formation has used before.
    /// MH: "when monster topples, hammer goes to head, cutter goes to tail."
    pub cooperation_patterns: Vec<CooperationPattern>,

    /// Vocabulary for structured communication.
    /// Siege: room names. MH: monster part names.
    pub shared_vocabulary: HashMap<String, VocabularyEntry>,

    /// Formation-specific conventions that emerged from experience.
    /// "In this formation, agent A usually handles X while agent B handles Y."
    pub conventions: Vec<Convention>,
}

pub struct CooperationPattern {
    pub trigger: PatternTrigger,
    pub participants: Vec<RoleInPattern>,
    pub success_count: u32,
    pub failure_count: u32,
    pub last_used: Instant,
}

pub struct Convention {
    pub description: String,
    pub established_by: Vec<AgentId>,
    pub strength: f32, // how consistently this convention has been followed
```

}

21.3 Design Principles

The L4D/Overcooked lesson: wherever possible, leverage existing shared knowledge rather than building from scratch. If agents are working with GitHub, the “cultural preload” is Git conventions. If agents are working with Discord, the preload is messaging platform conventions. The Siege lesson: shared vocabulary develops over time and becomes essential for efficient coordination. New agents joining an established formation should receive the accumulated vocabulary. The MH/DRG lesson: repeated successful cooperation builds pattern recognition that reduces future communication overhead. The mental model should persist across formation instances when the same agents work together again.

22. Tempo & Pacing

Constant high intensity burns agents out. Constant idleness wastes capacity.
Effective formations alternate between work peaks and recovery valleys.

22.1 Pacing Patterns Across All Games

Left 4 Dead — Adaptive Dramatic Pacing. The AI Director tracks estimated “emotional intensity” of survivors. “Constant, relentless action is wearing for the gamer — but too many long, slow gaps and the player gets bored. The solution is unpredictability.” The Director alternates between peaks (hordes, Tank encounters) and valleys (quiet traversal, safe rooms). (Source: AusGamers GDC 2009 coverage.)

Total War — Fatigue-enforced rest cycles. Units that fight continuously become exhausted — they fight slower, break morale faster, move sluggishly. You **MUST** rotate units out of engagement to recover stamina. This creates natural work/rest cycles without any top-down scheduling. The fatigue is a LOCAL property that changes behavior automatically.

Siege — Timer-enforced acceleration. The 3-minute round timer creates a mandatory pacing arc. First 90 seconds: slow, methodical (droning, intel gathering). Last 30 seconds: frantic execution. The timer forces prioritization between preparation and action. Teams that spend too long on intel have no time for the push. Teams that rush without intel die to prepared defenders.

Patapon — BPM as pacing. The music’s tempo literally controls how fast everything happens. Fever mode shifts the music, signaling the pacing change. The rhythm IS the pacing mechanism.

Overcooked — Disruptions as pacing resets. Duncan found that “disruptions” —

earthquakes, moving counters, sliding kitchens — were more effective than order pressure for pacing. They force a pause-and-adapt moment. "Suddenly players were having to really concentrate on what every member of the team was doing and stay in constant communication." (Source: Game Design Deep Dive.) The disruption breaks the current tempo and forces the team to re-establish coordination.

Helldivers 2 — Extraction as pacing climax. Missions build toward extraction — the final sprint to the shuttle under maximum enemy pressure. The pacing arc: deploy → spread out (low pressure) → objectives (building pressure) → extraction call (maximum pressure) → extract (release). Pilestedt's emphasis on fundamentals over features means the pacing arc is felt, not managed through UI.

Monster Hunter — Monster behavior as pacing. The monster's behavior creates natural pacing. It attacks (high pressure) → it exhausts and drools (recovery window) → it enrages (higher pressure) → it limps to a new area (transition/rest). The team's pacing follows the monster's state, not a timer.

Deep Rock Galactic — Swarm waves as pacing. Mission Control announces incoming swarms. The team has moments of calm exploration punctuated by intense defensive episodes. Resupply calls create gathering moments (convergence → coordination → resupply → disperse).

22.2 Pacing Architecture

```
// crates/springtale-bot/src/cooperation/pacing.rs

pub enum PacingPhase {
    /// Low intensity. Information gathering, preparation.
    /// L4D: quiet traversal. Siege: drone phase. DRG: exploration.
    Preparation { started: Instant },

    /// Building intensity. Work in progress, some pressure.
    /// MH: engaging the monster. Overcooked: orders coming in.
    Active { intensity: f32, started: Instant },

    /// Peak intensity. Maximum output, high fuel consumption.
    /// L4D: Tank encounter. Helldivers: extraction. Siege: execute.
    Peak { intensity: f32, fuel_rate: f32, started: Instant },

    /// Mandatory rest. Recovery, consolidation.
    /// Total War: fatigued units resting. L4D: safe room.
    /// DRG: resupply convergence.
    Recovery { remaining: Duration },

    /// Disruption. External event forces re-coordination.
```

```

    /// Overcooked: kitchen shift. MH: monster area transition.
    Disruption { event: DisruptionEvent },
}

pub struct PacingManager {
    pub current_phase: PacingPhase,
    pub cumulative_intensity: f32,    // tracks overall work done
    pub time_since_last_recovery: Duration,
    pub disruption_count: u32,

    // L4D Director-inspired thresholds
    pub peak_duration_max: Duration, // force recovery after this
    pub recovery_duration_min: Duration, // don't rush recovery
    pub intensity_ceiling: f32,        // max sustainable intensity
}

```

22.3 Design Principles

The L4D Director lesson: formations should NOT maintain constant peak intensity. The pacing manager should detect sustained high-intensity output and insert recovery phases. The Total War lesson: agents that work without rest degrade — their error rate increases and their capability decreases. The cadence system (§5) can modulate tick rate based on pacing phase — faster ticks during Peak, slower during Recovery. The Overcooked lesson: planned disruptions can be beneficial — they force re-coordination and prevent formations from falling into suboptimal ruts. The Siege lesson: external time pressure (constraints from §3.3) creates a natural pacing arc that teams must manage.

23. Specialization vs Generalization

The most effective agents do their primary job at full capacity AND provide cooperative benefits as a byproduct. Pure specialists and pure generalists are both suboptimal.

23.1 Specialization Patterns Across All Games

Monster Hunter — The backseat dooter problem. The MH community rejects players who abandon their primary weapon role to become full-time healers. “Don’t be a backseat dooter” — a Hunting Horn player who sits in the back playing healing songs instead of attacking is less valuable than one who attacks the monster’s head (dealing stun damage, contributing DPS) while healing through combat melodies. A player who does 100% damage AND 50% healing is worth far more than one doing 0% damage and 100% healing. (Source: Steam community discussions on HH support builds.)

Deep Rock Galactic — Unique capability + shared baseline. Every dwarf can mine, shoot, revive, and deposit minerals. But each class has ONE thing nobody else can do: Scout's grapple, Engineer's platforms, Driller's tunnels, Gunner's zipline/shield. Ghost Ship wanted "all teammates on even footing regardless of experience." (Source: Wikipedia.) Specialization is in the UNIQUE capability, not in the exclusion of general capabilities.

Siege — Gadget specialization, gunplay generalization. Operators are heavily specialized in gadgets (Thermite's exothermic charge, Mute's signal disruptors), but the universal mechanics — shooting, movement, destruction — are identical for all operators. A Thermite player with bad aim is less valuable than a Sledge player with great aim, even though Thermite's gadget is "more important." General competence trumps specialized capability.

It Takes Two — Forced asymmetric specialization. Each chapter gives each player completely different tools. You CAN'T use your partner's tool. But both tools are equally necessary and equally powerful. Neither player is "support." Neither is "carry." The asymmetry is in TYPE, not in IMPORTANCE.

Army of Two — Specialization through equipment, not role. Weapon customization biases the aggro role — suppressors for flanking, loud weapons for drawing fire. But any player can switch playstyle mid-mission by changing weapons. Specialization is a tendency, not a lock.

Left 4 Dead — No specialization at all. All four survivors are mechanically identical. The only asymmetry is which weapons each player happens to pick up. Cooperation emerges entirely from situational role-taking, not from designed specialization. This works because the threats (special infected) create transient, emergent roles — whoever is closest to the Smoker frees the grabbed survivor.

Overcooked — Station affinity, not station assignment. Duncan designed kitchens where no station is permanently assigned. Players gravitate toward stations based on proximity and what they see their partner doing. But disruptions force role changes — "players were being forced to change their roles midway through a round." (Source: Game Design Deep Dive.) Specialization is momentary and fluid.

23.2 Design Principles

Agents should have ALL general formation capabilities (awareness broadcasting, tick reporting, environment reading, handoff participation) PLUS unique capabilities from their connector type. An agent should never sacrifice its primary capability to perform cooperative functions. The Monster Hunter Hunting Horn is the design ideal: an agent whose primary work (attacking) naturally produces cooperative benefits (buffs/healing) for neighbors. The DRG model is the structural ideal: shared baseline + unique specialization. The L4D model shows that in some formation types, NO specialization is correct — all

agents are interchangeable and roles are purely situational.

The architectural implication: the `role_hint` in the composer (§3.1) should bias, not mandate. And the dynamic capability binding (§16) should ensure agents retain their general formation capabilities even when their specialized capability is exhausted or transformed.

24. Sacrifice & Covering

Distinct from recovery (§18), which helps agents already in trouble. Sacrifice is an agent deliberately accepting cost BEFORE failure occurs to benefit the formation.

24.1 Sacrifice Patterns Across All Games

Total War — Screening force. A cheap unit (Ashigaru) holds a position it's expected to lose, buying time for elite units to maneuver. The screening unit is expected to break and route. Its value is the TIME it buys, not the damage it deals. The player accepts the loss of the unit as a calculated investment in formation-level advantage.

Siege — Entry fragging. The entry fragger role is defined by accepting a high personal death rate to create opportunities for the team. The first player through a door will often trade kills (die but kill the defender), opening the site for teammates. The individual outcome is negative. The team outcome is positive.

Monster Hunter — Aggro drawing. Players routinely draw a monster's attention away from a teammate who's healing or sharpening. They accept hits they could have avoided to buy their teammate safety. There's no game mechanic for this — it's purely emergent from reading the situation and choosing to absorb cost for the team.

Army of Two — Feigned death. A player leaves themselves genuinely vulnerable (crawling, limited visibility, minimal defense) to give their partner an undetected flanking position. The "feigned death" player has shifted all aggro to their partner AND made themselves defenseless — double sacrifice for a tactical advantage.

Helldivers 2 — Self-bombing. Players call orbital strikes on their own position when overwhelmed, killing themselves along with the enemies, trusting that a teammate will reinforce them. This only works because the game's recovery system (reinforce stratagem) makes death temporary. The sacrifice calculation: certain death now + teammate reinforce later > probable team wipe from being overwhelmed.

Deep Rock Galactic — Gunner's shield as covering. Deploying the shield consumes a limited resource (shield charges). The Gunner spends their own resource to protect the

formation. The shield can be used offensively (protect during an objective completion), defensively (cover a revive), or for retreat (shield the extraction point). Each use costs a charge that the Gunner can't use for themselves later.

Left 4 Dead — Body blocking. A healthy survivor can physically position themselves between a threat and a wounded teammate, absorbing damage. The AI Director's special infected design mechanically enables sacrifice — you CAN free a Smoked teammate by shooting the tongue, taking the Smoker's attention.

Patapon — Defend rhythm as formation-wide sacrifice. Switching to CHAKA CHAKA PATA PON (defend) when the formation could be attacking means accepting zero offensive output in exchange for damage prevention. The entire formation sacrifices output for survivability. This is an orchestrator decision (intent change), but the individual Patapons' defensive behaviors — Tatepons raising shields, Yumipons ceasing fire — are cooperative sacrifice.

Overcooked — Station covering. When your partner is overwhelmed at their station, you leave YOUR station (accepting that your pipeline stalls) to help them. The cost is explicit: your dishes stop being prepared while you cover theirs. The decision is local: you see them drowning and you move to help, knowing your own station will back up.

Divinity OS2 — Healing over damage. Every Action Point spent on a healing spell is an Action Point not spent on damage. Choosing to heal a teammate is a direct sacrifice of offensive capability. The Necromancy exception (lifesteal from damage) is valuable precisely because it breaks this trade-off.

24.2 Sacrifice Architecture

```
// crates/springtale-bot/src/cooperation/sacrifice.rs

pub enum SacrificeType {
    /// Accept damage/cost to protect another agent.
    /// MH aggro drawing, L4D body blocking, DRG shield.
    Covering {
        sacrificer: AgentId,
        beneficiary: AgentId,
        cost_to_sacrificer: SacrificeCost,
        benefit_to_beneficiary: BenefitDescriptor,
    },

    /// Accept task degradation to assist another agent's task.
    /// Overcooked station covering, Siege entry fragging.
    TaskDiversion {
        sacrificer: AgentId,
```

```

        abandoned_task: TaskDescriptor,
        assumed_task: TaskDescriptor,
        formation_net_benefit: f32, // must be positive to justify
    },

    /// Accept individual destruction for formation benefit.
    /// Total War screening force, Helldivers self-bombing.
    Expendable {
        sacrificer: AgentId,
        expected_recovery: Option<RecoveryAction>, // Helldivers: reinforce
        formation_benefit: BenefitDescriptor,
    },

    /// Spend own resources on formation infrastructure.
    /// DRG Gunner shield, Patapon defend rhythm.
    ResourceInvestment {
        investor: AgentId,
        resource_spent: ResourceDescriptor,
        infrastructure_created: InfrastructureDescriptor,
        beneficiaries: Vec<AgentId>,
    },
}

pub struct SacrificeCost {
    pub fuel_cost: FuelBudget,
    pub capability_reduction: Vec<CapabilityDecl>, // what you lose
    pub vulnerability_increase: f32,
    pub duration: Duration,
}

```

24.3 Sacrifice Decision Framework

An agent evaluates sacrifice using the attention economy (§9) and awareness system (§8):

1. **Net positive check.** Does the formation's total output improve from the sacrifice? A Total War screening force that buys 30 seconds for the main army to flank is net positive even though the screen is destroyed.
2. **Recovery path check.** Is there a way back from the sacrifice? Helldivers self-bombing works because reinforce exists. A sacrifice with no recovery path should require higher net benefit to justify.
3. **Capability preservation check.** Does the sacrifice eliminate a unique capability the formation needs? Losing a generalist is less costly than losing a specialist. The DRG Driller self-sacrificing is more costly than an extra Scout because nobody else can drill.

4. **Momentum impact check.** Does the sacrifice risk breaking formation momentum (§7)?
An agent going down during Fever tier could crash the formation to Cold. The sacrifice might not be worth the momentum loss even if the immediate tactical benefit is positive.

The key insight from Total War's screening force pattern: sometimes the most cooperative thing an agent can do is accept its own degradation to create space for other agents to succeed. This must be VOLUNTARY (the agent decides based on local awareness) not COMMANDED (the orchestrator ordering an agent to sacrifice). Orchestrated sacrifice is micromanagement. Cooperative sacrifice is mutual aid.

25. Module Summary

The complete cooperation module:

```
crates/springtale-bot/src/cooperation/
├─ mod.rs
├─ cadence.rs           # §5  - shared tick bus (external clock)
├─ formation.rs         # §6  - peer agent grouping + lifecycle
├─ momentum.rs         # §7  - coherence accumulator / Fever
├─ awareness.rs        # §8  - local neighbor perception
├─ attention.rs        # §9  - workload economy / aggro
├─ environment.rs      # §10 - shared mutable workspace
├─ consensus.rs        # §11 - weighted decision resolution
├─ commit.rs           # §12 - synchronized execution barriers
├─ interference.rs     # §13 - conflict detection
├─ transformation.rs   # §14 - role change on capability loss
├─ rally.rs            # §15 - cascade recovery
├─ capability.rs       # §16 - dynamic capability binding
├─ recovery.rs         # §18 - distress detection, mutual aid
├─ comms.rs           # §19 - multi-layer communication
├─ handoff.rs          # §20 - work product transfer
├─ mental_model.rs     # §21 - shared context accumulation
├─ pacing.rs           # §22 - work/rest cycle management
├─ sacrifice.rs        # §24 - deliberate self-cost for team benefit
```

Sections §17 (integration) and §23 (specialization vs generalization) are design principles that inform all modules rather than having their own module implementations. §23's principles are enforced through the composer (§3.1), capability binding (§16), and the convention that agents should never abandon their primary capability to perform cooperative functions.